Representação de Grafos

Estruturas de Dados



UnB | CIC

Definição

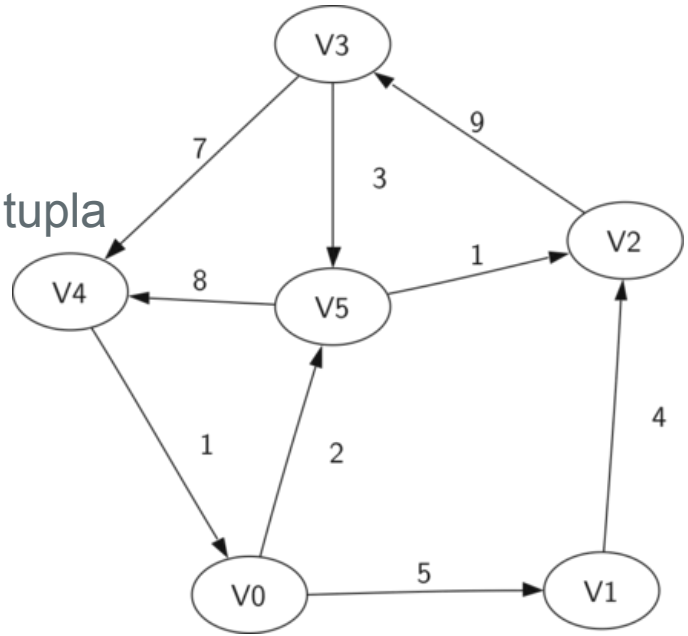
Um grafo pode ser representado por **G** onde **G = (V,E)**.

- **V** é um conjunto de vértices
- **E** é um conjunto de arestas.
- Cada aresta é uma tupla *(v,w)* onde $v,w \in$ **V**.
- Podemos adicionar um terceiro componente à tupla de arestas para representar o peso.

Definição

Um grafo pode ser representado por **G** onde **G = (V,E)**.

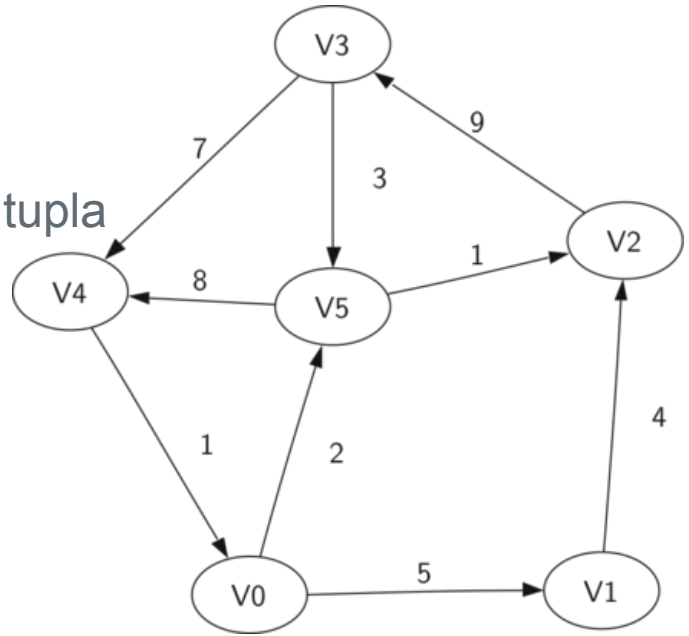
- **V** é um conjunto de vértices
- **E** é um conjunto de arestas.
- Cada aresta é uma tupla *(v,w)* onde *v*,*w* ∈ **V**.
- Podemos adicionar um terceiro componente à tupla de arestas para representar o peso.



Definição

Um grafo pode ser representado por **G** onde **G = (V,E)**.

- **V** é um conjunto de vértices
- **E** é um conjunto de arestas.
- Cada aresta é uma tupla *(v,w)* onde *v*,*w* ∈ **V**.
- Podemos adicionar um terceiro componente à tupla de arestas para representar o peso.

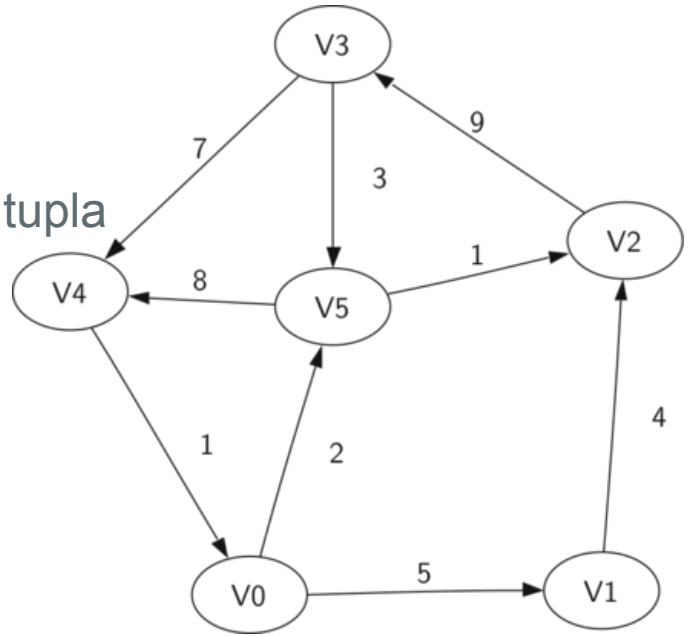


V = {V0, V1, V2, V3, V4, V5}

Definição

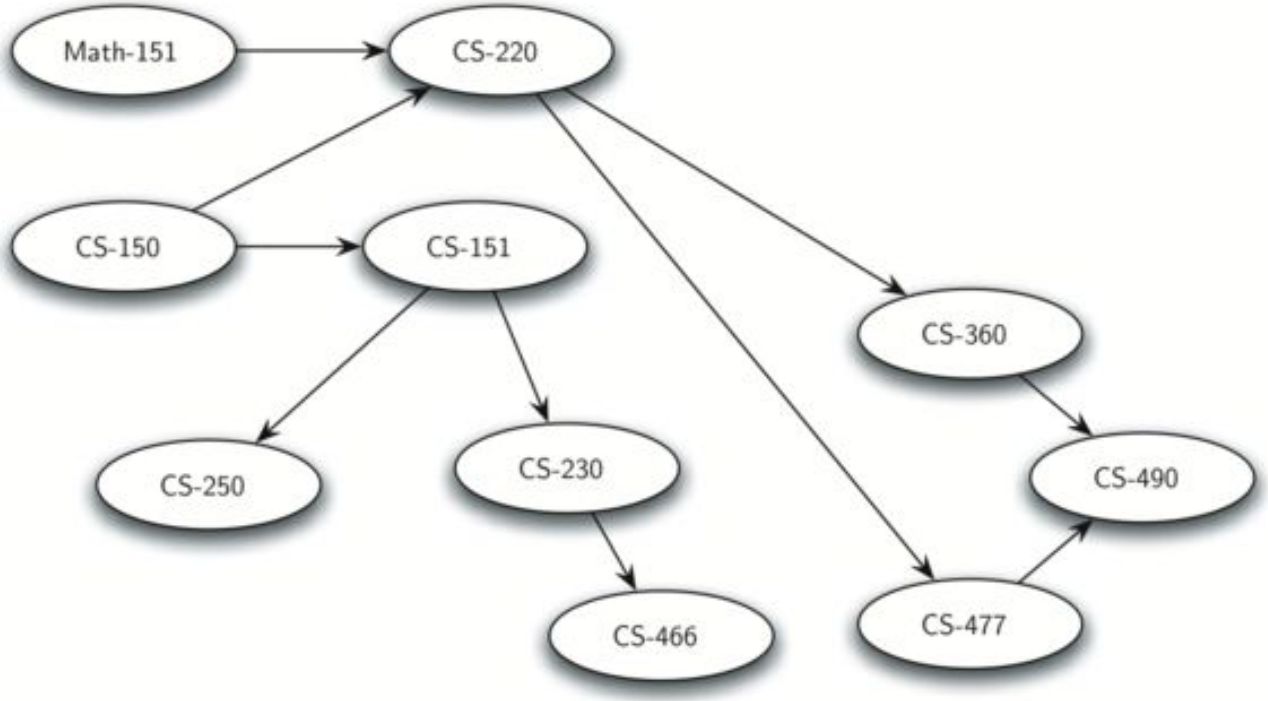
Um grafo pode ser representado por **G** onde **G = (V,E)**.

- **V** é um conjunto de vértices
- **E** é um conjunto de arestas.
- Cada aresta é uma tupla *(v,w)* onde *v*,*w* ∈ **V**.
- Podemos adicionar um terceiro componente à tupla de arestas para representar o peso.



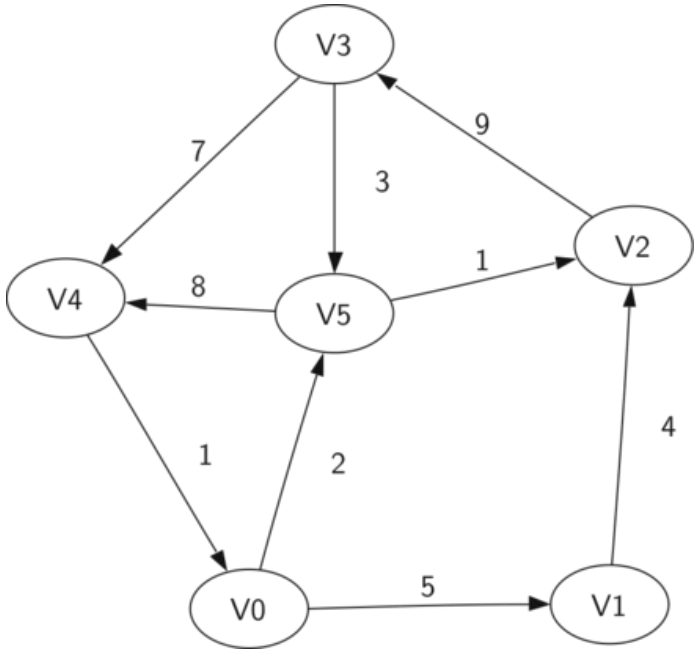
V = {V0, V1, V2, V3, V4, V5}
E = {(V0, V1, 5), (V1, V2, 4), (V2, V3, 9), (V3, V4, 7),
(V4, V0, 1), (V0, V5, 2), (V5, V4, 8), (V3, V5, 3), (V5, V2, 1)}

Exemplo: Dependência de Disciplinas



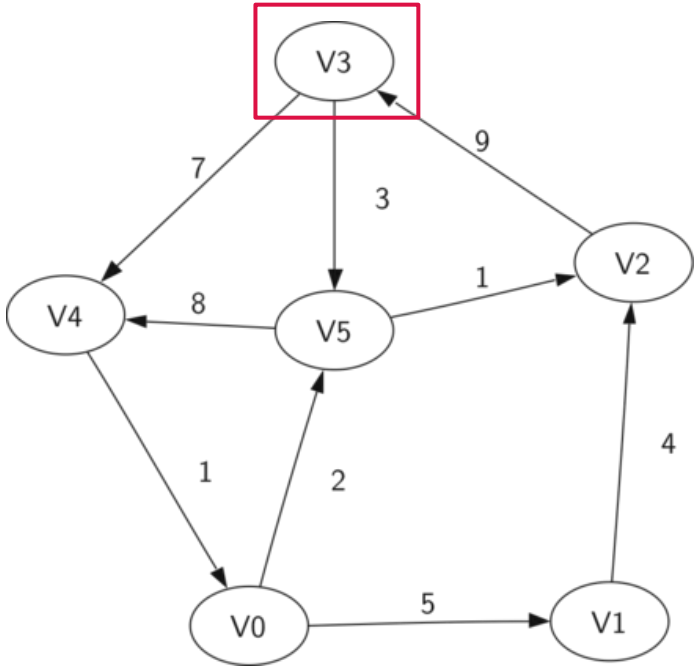
Vocabulário

- Vértice
- Aresta
- Peso
- Caminho
- Ciclo



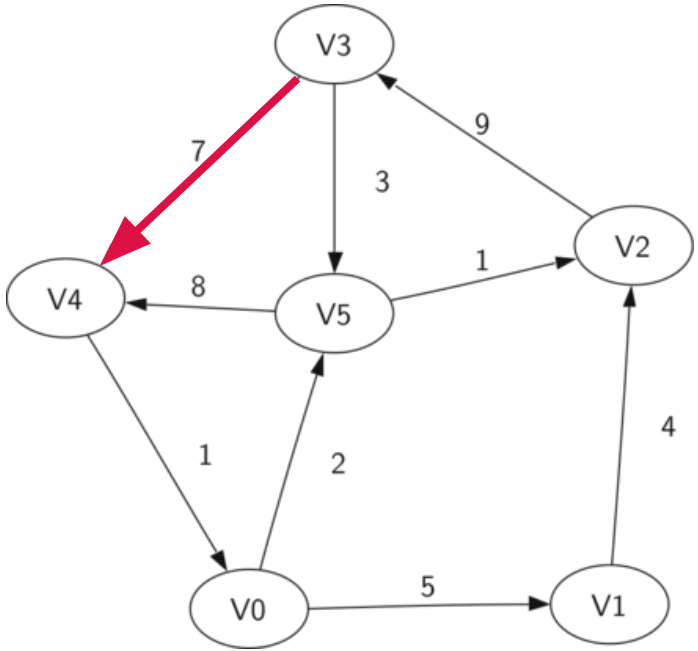
Vocabulário

- **Vértice**
- Aresta
- Peso
- Caminho
- Ciclo



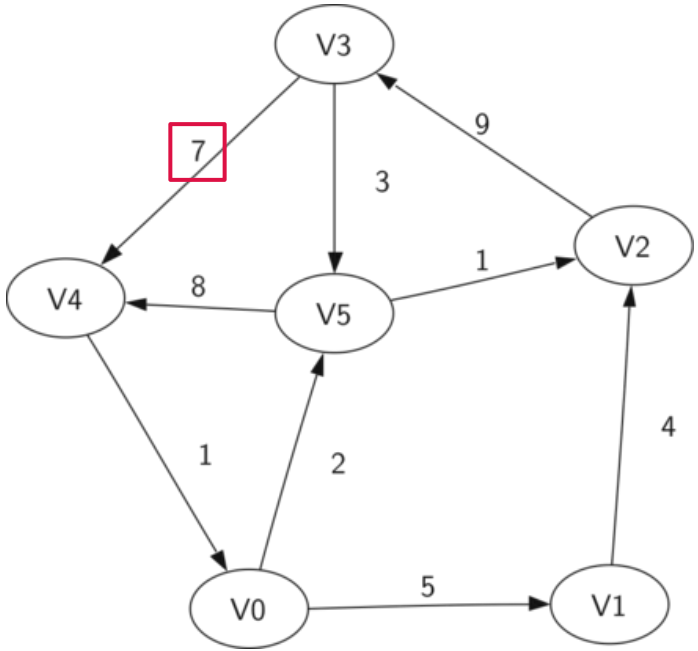
Vocabulário

- Vértice
- **Aresta**
- Peso
- Caminho
- Ciclo



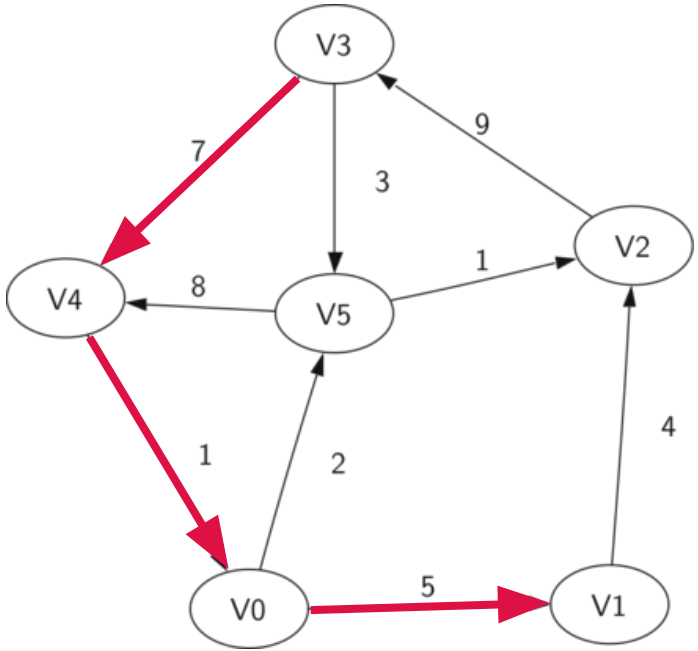
Vocabulário

- Vértice
- Aresta
- **Peso**
- Caminho
- Ciclo



Vocabulário

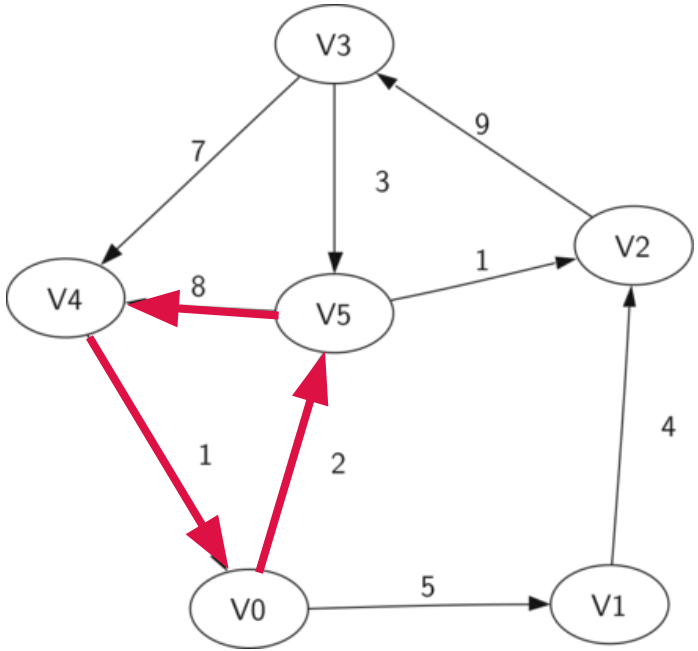
- Vértice
- Aresta
- Peso
- **Caminho**
- Ciclo




```
(V3, V4, V0, V1)
{(V3, V4, 7), (V4, V0, 1), (V0, V1, 5)}
```

Vocabulário

- Vértice
- Aresta
- Peso
- Caminho
- **Ciclo**



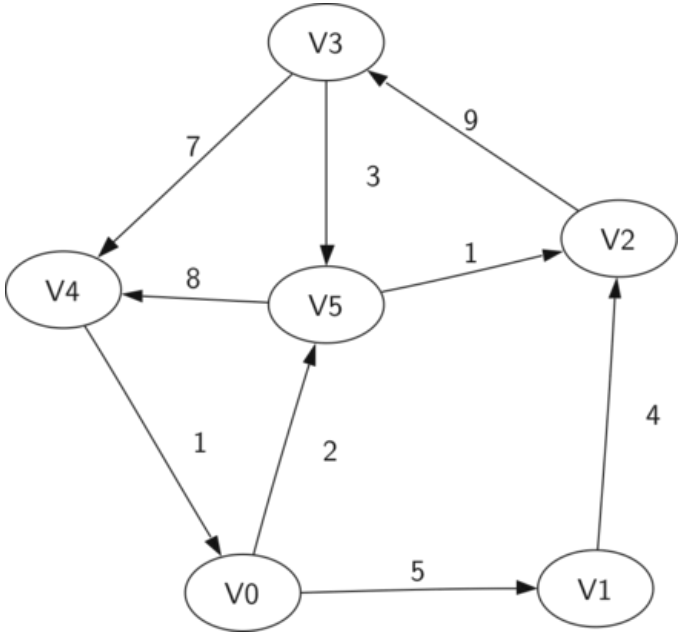

```
(V4, V0, V5)
{(V4, V0, 1), (V0, V5, 2), (V5, V4, 8)}
```

Tipo Abstrato de Dados Grafo

- `Graph()` cria um grafo vazio.
- `addVertex(vert)` adiciona uma instância de um vértice ao grafo.
- `addEdge(fromVert, toVert)` adiciona uma aresta direcionada entre dois vértices.
- `addEdge(fromVert, toVert, weight)` adiciona uma aresta direcionada e com peso entre dois vértices.
- `getVertex(vertKey)` procura um vértice com chave *`vertKey`* no grafo.
- `getVertices()` retorna a lista de vértices do grafo.
- `in` retorna `True` ou `False` se o vértice está ou não no grafo.

Representação por matriz

	V0	V1	V2	V3	V4	V5
V0		5				2
V1			4			
V2				9		
V3					7	3
V4	1					
V5			1		8	



Representação por matriz

```
class Graph:
    def __init__(self):
        self.vertices = []
        self.adj_matrix = []
```

Representação por matriz

```
class Graph:
    ...
    def addVertex(self, key):
        if key not in self.vertices:
            self.vertices.append(key)

            for row in self.adj_matrix:
                row.append(0)

            new_row = [0] * len(self.vertices)
            self.adj_matrix.append(new_row)
```

Representação por matriz

```
class Graph:
    ...
    def addEdge(self, fromVert, toVert, weight=0):
        if fromVert in self.vertices and toVert in self.vertices:
            linha = self.vertices.index(fromVert)
            coluna = self.vertices.index(toVert)
            self.adj_matrix[linha][coluna] = weight
```

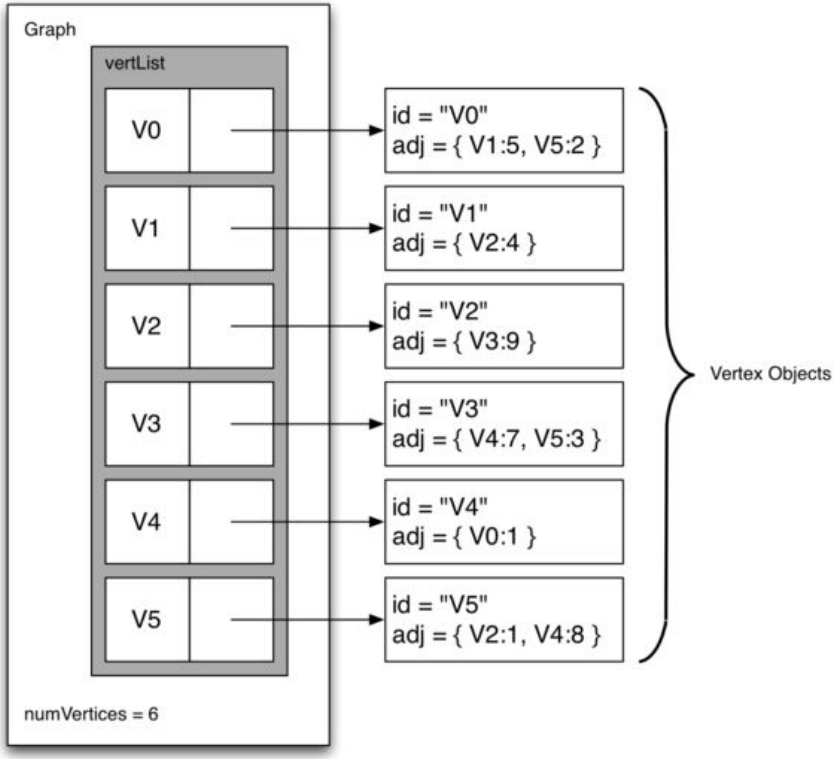
Representação por matriz

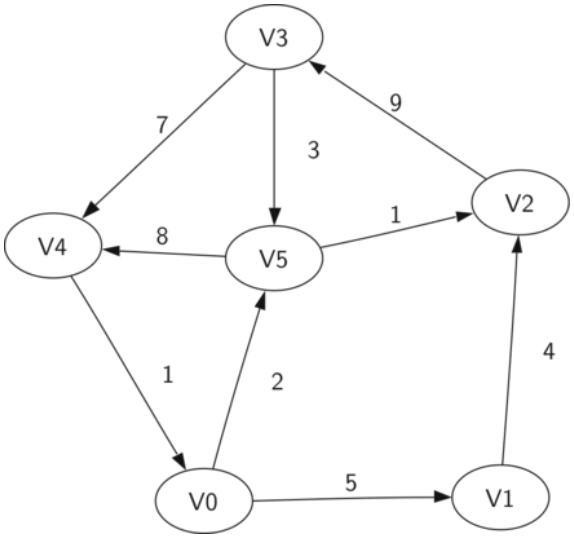
```
class Graph:
    ...
    def getVertex(self, vertKey):
        return vertKey if vertKey in self.vertices else None

    def getVertices(self):
        return self.vertices

    def __contains__(self, vertKey):
        return vertKey in self.vertices
```

Representação por listas de adjacências





Representação por listas de adjacências

```
class Vertex:
    def __init__(self, key):
        self.id = key
        self.connectedTo = {}

    def addNeighbor(self, nbr, weight=0):
        self.connectedTo[nbr] = weight

    def getConnections(self):
        return self.connectedTo.keys()

    def getId(self):
        return self.id

    def getWeight(self, nbr):
        return self.connectedTo[nbr]
```

Representação por listas de adjacências

```
class Graph:
    def __init__(self):
        self.vertList = {}
        self.numVertices = 0
```

Representação por listas de adjacências

```
def addVertex(self, key):
    self.numVertices = self.numVertices + 1
    newVertex = Vertex(key)
    self.vertList[key] = newVertex
    return newVertex

def getVertex(self, n):
    if n in self.vertList:
        return self.vertList[n]
    else:
        return None
```

Representação por listas de adjacências

```
def addEdge(self, f, t, weight=0):
    if f not in self.vertList:
        nv = self.addVertex(f)
    if t not in self.vertList:
        nv = self.addVertex(t)
    self.vertList[f].addNeighbor(self.vertList[t], weight)

def getVertices(self):
    return self.vertList.keys()
```

Representação por listas de adjacências

```
graph = Graph()

graph.addVertex('A')
graph.addVertex('B')
graph.addVertex('C')
graph.addVertex('D')

graph.addEdge('A', 'B')
graph.addEdge('B', 'C')
graph.addEdge('C', 'D')
graph.addEdge('D', 'A')
```

Resumo

- Um grafo é uma representação de elementos (vértices) e suas relações (arestas).
- A representação de um grafo pode ser por:
  - matriz (simples, mas talvez computacionalmente ineficiente).
  - listas de adjacências (complexa, mas talvez computacionalmente eficiente).
- Grafos são estruturas mais abstratas do que árvores.